SISTEMAS DE TIEMPO REAL

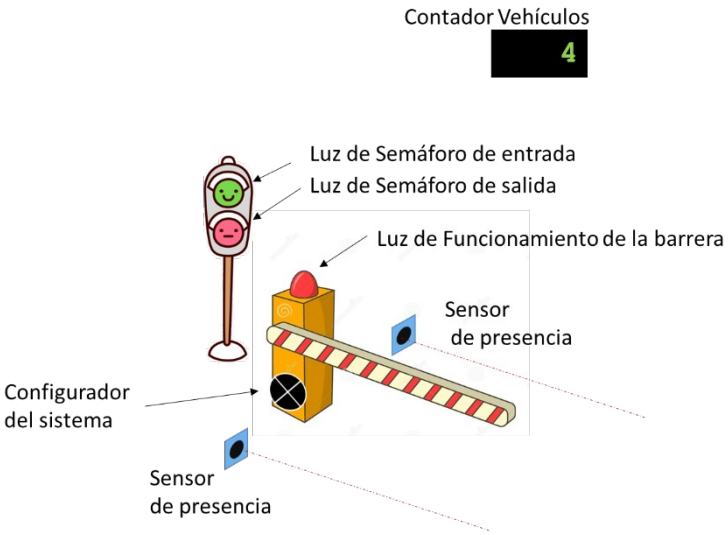
(Ejercicio Propuesto)

Se pretende diseñar e implementar un Sistema de Tiempo Real para el control de acceso a un parking.

El sistema cuenta una única barrera que controla el acceso y salida del parking.

Disponemos de los siguientes elementos (ver figura):

- Sensor de presencia indicando que un vehículo va a entrar al parking.
- Sensor de presencia indicando que un vehículo va a salir del parking
- 2 semáforos uno de entrada y otro de salida, indicando con luz verde el libre acceso de los vehículos y roja que no pueden pasar. Cuando un vehículo va a entrar (salir), el semáforo de salida (entrada) se debe poner en rojo (verde).
- Un motor que controla la barrera.
- Una luz que parpadea cuando la barrera se encuentra en movimiento.
- Un contador que indica en todo momento el número de vehículos en el interior del parking.
- Un configurador del sistema (potenciómetro) para establecer la temporización del parpadeo de la luz de la barrera.



El funcionamiento del sistema es el siguiente:

- Inicialmente los dos semáforos de paso se encuentran en verde.
- La entrada/salida de un vehículo se detecta por su sensor de presencia, que produce la siguiente secuencia:
  - 1.- Apaga el semáforo de salida/entrada,
  - 2.- Acciona la barrera para abrir, y
  - 3.- Activa la luz parpadeante de funcionamiento de la barrera.

  - Una vez que el vehículo entra/sale, hecho que se detecta mediante el sensor de presencia al otro lado de la barrera se produce la secuencia:
  - 1.- Vuelve a encender el semáforo de salida/entrada.
  - 2.- Acciona la barrera para bajar,
  - 3.- Apaga la luz parpadeante
  - 4.- Actualiza el número de vehículos en el interior de parking.
- En cualquier momento en el que el sistema esté en reposo (barrera bajada) se puede accionar un modo de configuración, en el que mediante un potenciómetro se puede ajustar la velocidad de parpadeo de la luz de la barrera, así como resetear el número de vehículos que están en el interior del parking.

Con el fin de implementar este sistema en la placa de desarrollo utilizada en clase se asume lo siguiente:

Los leds se van a utilizar para simular los semáforos, luz parpadeante y el contador. Concretamente se utilizará el **led 7** para el semáforo de entrada (encendido: verde, apagado: rojo). El **led 6** para el semáforo de salida. Los **leds 2,3,4** para implementar el contador en binario (se asume que no puede haber más de 7 coches en el interior). Finalmente el **led 0** se utilizará como luz de parpadeo.

LED	7	6	5	4	3	2	1	0
Patilla	PC.3	PC.4	PD.4	PD.5	PD.7	PB.0	PB.1	PB.2

La barrera está accionada por un servomotor controlado por una onda PWM de frecuencia base de 50 Hz. Con una señal de ancho positivo de 1 ms, la barrera está cerrada y con un ancho positivo de 1.5 ms, la barrera está abierta. Utiliza el timer 1 del Atmega328p y el modo FastPWM (modo 14), utilizando el preescalado más pequeño posible. La señal de salida de la onda PWM se produce en PB1, donde podrás conectar un osciloscópio y si quieres ver un motor en funcionamiento, podrás conectar un servo del laboratorio.

Utiliza el timer 0 para temporizar el parpadeo de la luz de la barrera. Inicialmente debe parpadear cada 0.1 segundos.

Utiliza los botones de la placa para simular los sensores de presencia: Botón A simula el sensor de entrada y el Botón B el de salida. Utiliza igualmente el Botón C para activar el modo de configuración, en el que se podrá leer del potenciómetro para ajustar el tiempo de parpadeo, desde 0.1 segundo hasta 1 segundo en función de la posición analógica leída. Con una nueva pulsación del Botón C se vuelve a la situación inicial.

Considera la casuística lógica del sistema, como por ejemplo que no pueden salir más coches de los que hay, no pueden entrar coches si el parking está lleno, no pueden entrar/salir dos coches seguidos, etc.

Una vez el comportamiento sea el deseado, puedes simularlo en el entorno virtual[1]. Para ello sigue los siguientes pasos:

- Carga el proyecto de MPLAB del CV. Este proyecto incluye una librería para la comunicación por puerto serie y una cabecera que configura la escena.

- Modifica tu código para que se ajuste al esquema de la figura. Es decir, durante la fase de inicialización, estando protegido por cli/sei, añade como última línea la función `Init_Coppelia_Interface()`. Después el sistema se queda en un bucle infinito recibiendo información de la escena virtual.

```
int main(void)
{
     cli();

     //Inicializaciones

     Init_Coppelia_Interface();

     sei();


     while(1)
        {

              Receive_from_Coppelia();
     }

}
```

Posteriormente lanza CoppeliaSim y carga la escena proporcionada. Para indicar a Coppelia en qué puerto está conectado el arduino, crea un fichero de texto llamado "port.txt" en la misma carpeta donde se encuentra la simulación.

En Linux, el contenido de dicho fichero será típicamente:
/dev/ttyACM0

En Windows, será:
COMx

donde x indica el puerto asignado por windows.

1 Sólo podrás simular el paso de vehículos, conteo de los mismos y encendido de semáforos. El parpadeo del led y su actuación con el potenciómetro no está simulado.

Hemos detectado algunas diferencias en el comportamiento del simulador, especialmente para windows. Así que podrás ver que hay dos escenas, una para cada sistema operativo. Adicionalmente para windows hemos comprobado que es necesario resetear arduino una vez lanzada la simulación.